Agora — Phased Implementation Plan

> Derived from PRD v1.0 and Architecture Design Document Each phase builds on the previous, introducing new platform capabilities incrementally. Key principle: **validate the platform core with the simplest mode first, then add complexity.**

Phase Overview

Phase	Name	Core Unlocks	New Modes	Target
1	Roundtable MVP	Agent, Room, FreeForm Flow, LLM, basic UI	Roundtable Debate	1 week
2	Werewolf	Channel (info isolation), StateMachine Flow, Structured Output	Werewolf	2 weeks
3	UX Polish	Room View visualization, Persona Editor, Human Players, Spectator	— (enhance existing)	1-2 weeks
4	Script Kill	Long-term Memory, Clue/Evidence system, Branching Narrative	Script Kill	2-3 weeks
5	TRPG	GM Agent, Dice system, Narrative generation, Character growth	TRPG	3 weeks

Phase	Name	Core Unlocks	New Modes	Target
6	Platform	Custom Mode SDK, Agent Marketplace, Replay, Hierarchical Flow	Custom	2 weeks

Phase 1: Roundtable Debate — Validate Platform Core

Goal: N agents (each using a different LLM model) debate a user-given topic in a group chat UI. Proves that Room, Agent, FlowController, and multi-model LLM integration work end-to-end.

Step 1.1: Project Scaffolding

- [ ] Initialize Turborepo monorepo
- [ ] Create `apps/web` with Next.js 15 (App Router)
- [ ] Create `packages/core` (empty, with tsconfig)
- [ ] Create `packages/llm` (empty, with tsconfig)
- [ ] Create `packages/modes` (empty, with tsconfig)
- [ ] Create `packages/shared` (types, constants)
- [ ] Configure Tailwind + shadcn/ui in `apps/web`
- [ ] Verify `turbo dev` runs the frontend

Deliverable: `turbo dev` serves a blank Next.js app.

Step 1.2: Core Abstractions (packages/core)

- [ ] `Message` type — id, sender, content, channel, timestamp, metadata
- [ ] `Agent` interface — reply(), observe(), persona config, model binding
- [ ] `AIAgent` implementation — wraps Vercel AI SDK `generateText`
- [ ] `Room` class — create, add/remove agents, lifecycle (waiting → active → ended)

- [ ] `FreeFormFlow` — simplest FlowController: round-robin with optional max rounds
- [ ] `EventBus` — typed event emitter for UI subscription (messageCreated, agentReplied, roundChanged)

Deliverable: Can create a Room, add 3 AIAgents, run a FreeFormFlow, and see messages in console.

Step 1.3: LLM Integration (packages/llm)

- [ ] Vercel AI SDK wrapper — `createAgentModel(provider, modelId, apiKey?)`
- [ ] Multi-provider support: Anthropic (Claude), OpenAI (GPT), Google (Gemini)
- [ ] Streaming support via `streamText`
- [ ] Error handling + retry logic
- [ ] API key management (env vars for MVP)

Deliverable: Can call Claude, GPT, and Gemini through a unified interface.

Step 1.4: Roundtable Mode (packages/modes/roundtable)

- [ ] Mode definition: roles = [{id: "debater", count: [2, 8]}]
- [ ] Flow config: FreeFormFlow with configurable rounds (default 3-5)
- [ ] System prompts: debater persona template
- [ ] Optional: vote for best argument at end of each round
- [ ] Moderator logic: announce topic, call each agent in turn, summarize

Deliverable: Roundtable mode registered and runnable via Room.

Step 1.5: Frontend — Basic Chat UI (apps/web)

- [ ] Home page: "Create Room" button
- [ ] Room creation form: topic, number of agents, model selection per agent
- [ ] Room page: group chat UI with message bubbles
- Agent name + model badge on each bubble
- Different colors per agent

- Auto-scroll, typing indicator
- Markdown rendering in messages
- [ ] API routes: POST /api/rooms, GET /api/rooms/:id, POST /api/rooms/:id/start
- [ ] Socket.io integration: real-time message streaming from server to client

Deliverable: User creates a room in browser, picks a topic and 3-5 agents with different models, hits "Start", and watches them debate in real-time.

Step 1.6: First Deploy

- [ ] Deploy to Vercel
- [ ] Environment variables configured (API keys)
- [ ] Verify end-to-end flow works in production

Acceptance Criteria (Phase 1): - [ ] User can create a room with a topic - [ ] User can add 2-8 AI agents, each with a different model - [ ] Agents debate the topic in a group chat, each maintaining their persona - [ ] Messages stream in real-time to the browser - [ ] Each agent's model is visually indicated (badge) - [ ] A debate completes 3-5 rounds without errors - [ ] Deployed and accessible via public URL

Phase 2: Werewolf — Channel Isolation + State Machine

Goal: Full werewolf game with night/day phases, private wolf channel, voting, and elimination. Proves Channel isolation and StateMachine flow work.

Step 2.1: Channel System (packages/core)

- [ ] `Channel` class — id, subscribers, autobroadcast toggle
- [ ] Nested channels — outer (all players) + inner (wolf-only)
- [ ] Visibility mask — message routing respects channel membership
- [ ] Dynamic subscribe/unsubscribe (dead players removed)
- [ ] Channel-aware EventBus — only emit to clients who should see

Step 2.2: StateMachine FlowController (packages/core)

- [ ] `StateMachineFlow` — generic state machine with phases, transitions, actions
- [ ] Phase definition: { id, allowedChannels, actions, nextPhase, condition }
- [ ] Action types: discuss (sequential), vote (fanout), announce (broadcast)
- [ ] Transition conditions: vote result, timer, custom predicate

Step 2.3: Structured Output (packages/core)

- [ ] `createDecisionSchema(type, context)` — generates Zod schema dynamically
- [ ] Vote schema: `z.object({ target: z.enum(alivePlayers) })`
- [ ] Discussion schema: `z.object({ message: z.string(), reachAgreement: z.boolean() })`
- [ ] Vercel AI SDK `generateObject` integration for constrained decisions

Step 2.4: Werewolf Mode (packages/modes/werewolf)

- [ ] Roles: werewolf (2-3), villager (3-4), seer (1), witch (1)
- [ ] Phases: Night (wolf discuss → wolf vote → seer check → witch act) → Day (announce → discuss → vote → eliminate)
- [ ] Channel config: `public` (day), `werewolf` (night), `seer-result` (private), `witch-action` (private)
- [ ] Win conditions: all wolves dead OR wolves >= villagers
- [ ] Role assignment: random shuffle at game start
- [ ] Detailed prompts per role (wolf strategy, seer investigation, witch dilemma)

Step 2.5: Frontend — Game UI

- [ ] Phase indicator (night/day with visual transition)
- [ ] Vote panel (shows alive players, collects votes)
- [ ] Role reveal (private, only shown to the agent's owner)
- [ ] Elimination announcement with animation
- [ ] Win/loss screen

- [ ] Night overlay (dim screen during wolf phase if spectator)

Acceptance Criteria (Phase 2): - [ ] 6-9 agent werewolf game runs to completion - [ ] Wolves can discuss privately — villagers NEVER see wolf chat - [ ] Seer gets private check result — no one else sees it - [ ] Votes are collected independently (no one sees others' votes before reveal) - [ ] Win condition is correctly detected and game ends - [ ] Spectator can see everything (god mode toggle) - [ ] Adding werewolf mode did NOT require changes to packages/core

Phase 3: UX Polish — Make People Want to Share

Goal: Elevate the visual experience to Accio-Work-level polish. Make sessions screenshot-worthy.

Step 3.1: Room View Visualization

- [ ] 2D room scene with agent avatars positioned around a table/circle
- [ ] Speech bubbles that appear and fade (borrowed from evotraders RoomView)
- [ ] Agent avatars with model logos and status indicators
- [ ] Smooth animations (Framer Motion) for speaking, voting, elimination

Step 3.2: Persona Editor

- [ ] Rich persona creation form: name, avatar, personality description, background
- [ ] AI auto-enrich: user writes 2-3 sentences, AI expands into full character
- [ ] Persona preview: see how the agent would respond to a sample prompt
- [ ] Persona library: save and reuse personas across rooms

Step 3.3: Human Player Support

- [ ] `HumanAgent` implementation in packages/core
- [ ] When it's a human's turn, show input UI instead of auto-generating
- [ ] Human can vote, discuss, use special abilities (if game mode)

- [ ] Mixed rooms: some agents AI, some human

Step 3.4: Spectator Mode

- [ ] Join any public room as spectator (no participation, just watch)
- [ ] God mode toggle: see all channels including private ones
- [ ] Live viewer count
- [ ] Shareable room URL

Acceptance Criteria (Phase 3): - [ ] Room View looks polished enough that users screenshot and share it - [ ] Human can play alongside AI agents in werewolf - [ ] Personas created in one room can be reused in another - [ ] Spectators can watch live games without affecting them

Phase 4: Script Kill — Deep Narrative + Memory

Goal: Full murder mystery game with private clues, investigation phases, and branching narrative. Introduces long-term memory.

Step 4.1: Session Memory Compression (packages/core)

- [ ] `SessionMemory` class with LLM-powered compression
- [ ] When messages > threshold (e.g., 40), compress old messages into summary
- [ ] Keep summary + recent N messages in context window
- [ ] Use cheap model (Haiku) for compression

Step 4.2: Agent Long-Term Memory (packages/core)

- [ ] `AgentLongTermMemory` with pgvector
- [ ] record(event) → embed → store in Postgres
- [ ] retrieve(query) → cosine similarity search → inject into prompt
- [ ] reflect() — periodic self-reflection (Stanford Generative Agents pattern)

Step 4.3: Clue/Evidence System

- [ ] `Clue` type — content, visibility (who has seen it), importance
- [ ] Private clue distribution — each character gets unique clues at game start
- [ ] Clue discovery — agents can find new clues during investigation phases
- [ ] Clue sharing — agents can choose to share or withhold clues in discussion

Step 4.4: Script Kill Mode (packages/modes/script-kill)

- [ ] Script template system: define characters, clues, timeline, true culprit
- [ ] Phases: Introduction → Investigation Round 1-3 → Discussion → Accusation → Reveal
- [ ] Branching: investigation choices affect what clues are found
- [ ] AI script generation: given a theme, generate a complete murder mystery
- [ ] Deep persona prompts: each character has background, secrets, motivations

Acceptance Criteria (Phase 4): - [ ] Complete murder mystery runs with 4-6 agents - [ ] Each agent has unique private clues that others cannot see - [ ] Agents remember and reference earlier conversations (memory compression works) - [ ] At least one AI-generated script is playable and solvable - [ ] Agents with long-term memory show improved play over multiple games

Phase 5: TRPG — Open-World Narrative

Goal: AI Game Master runs a tabletop RPG session with AI/human players. Most complex mode.

Step 5.1: GM Agent

- [ ] Specialized agent type with world-state awareness
- [ ] Narrative generation: describe scenes, NPCs, consequences
- [ ] Rules engine integration: D&D 5e-lite skill checks
- [ ] Dynamic difficulty adjustment based on player engagement

Step 5.2: Dice System

- [ ] `DiceRoll` type — notation (d20, 2d6+3), result, context
- [ ] Skill check flow: player declares action → GM sets DC → roll → GM narrates outcome
- [ ] Visual dice animation in UI

Step 5.3: Character System

- [ ] Character sheet: name, class, stats, inventory, backstory
- [ ] Character creation wizard (AI-assisted)
- [ ] Character growth: XP, level up, new abilities
- [ ] Persistent characters across sessions

Step 5.4: TRPG Mode (packages/modes/trpg)

- [ ] Semi-freeform flow: GM-guided with player agency
- [ ] Combat mode (structured) vs Exploration mode (freeform)
- [ ] NPC management: GM creates and voices NPCs
- [ ] Campaign persistence: save world state between sessions

Acceptance Criteria (Phase 5): - [ ] GM Agent runs a coherent 30-60 minute adventure - [ ] Players can take creative actions (not just menu choices) - [ ] Dice rolls affect outcomes meaningfully - [ ] Campaign state persists across sessions - [ ] GM maintains world consistency (NPCs remember past interactions)

Phase 6: Platform — Open Up

Goal: Let users create their own modes. Build the ecosystem.

Step 6.1: Custom Mode SDK

- [ ] Mode definition schema (JSON/YAML for non-developers)
- [ ] Mode builder UI: define roles, flow, channels, prompts visually

- [ ] Mode validation: ensure mode definition is complete and consistent
- [ ] Mode testing sandbox: run a mode in preview before publishing

Step 6.2: Agent Marketplace

- [ ] Public persona gallery: browse and fork community-created agents
- [ ] Tags, ratings, usage counts
- [ ] Featured agents/modes curation

Step 6.3: Enhanced Replay

- [ ] Record all events with timestamps
- [ ] Replay viewer: step through a session like a recording
- [ ] Export as shareable link or video-like format
- [ ] Highlight reel: AI-generated summary of best moments

Step 6.4: Hierarchical FlowController

- [ ] Leader-worker pattern (三省六部 style)
- [ ] Task delegation and aggregation
- [ ] Review gates (propose → review → approve/reject)
- [ ] Multi-level hierarchy support

Acceptance Criteria (Phase 6): - [ ] Non-developer can create a custom mode via UI - [ ] At least 5 community-created modes published - [ ] Replay of any session is watchable and shareable - [ ] Hierarchical flow enables "OPC company" style simulations

Technical Milestones (Cross-Phase)

Milestone	Phase	Description
First agent speaks	1.2	AIAgent.reply() produces a message via Vercel AI SDK
First multi-agent conversation	1.2	3 agents in a Room exchange messages via FreeFormFlow
First UI render	1.5	Messages appear as chat bubbles in the browser
First deploy	1.6	Public URL serves a working roundtable debate
First info isolation	2.1	Wolf chat is invisible to villagers
First structured decision	2.3	Agent votes for a real player name (not hallucinated)
First game completion	2.4	Werewolf runs to win/loss without manual intervention
First human + AI game	3.3	Human plays alongside AI agents
First memory compression	4.1	Long conversation compressed without losing key facts
First cross-session memory	4.2	Agent references a previous game's events
First AI-generated content	4.4	AI writes a playable murder mystery script
First GM narration	5.1	GM Agent describes a scene and reacts to player input
First custom mode	6.1	User creates a mode without writing TypeScript

Risk Mitigation

Risk	Impact	Mitigation
LLM costs spiral during multi-agent sessions	High	Cost tracking per session, configurable limits, use cheaper models for compression/roles
Agents break character or ignore game rules	Medium	Structured output (Zod) for decisions, strong system prompts, few-shot examples
WebSocket scaling with many concurrent rooms	Medium	Socket.io rooms for isolation, consider PartyKit for edge if needed
Vercel AI SDK doesn't support a needed provider	Low	SDK has provider interface; can add custom providers
Session state loss on server restart	Medium	Persist room state to Postgres, implement session restore
Prompt injection by human players	Medium	Separate system prompt channel, input sanitization, rate limiting

Dependencies & Decisions Log

Decision	Made	Rationale
TypeScript full-stack (not Python)	2026-04-13	UI is core value prop; single-stack efficiency; Vercel AI SDK covers LLM needs

Decision	Made	Rationale
Not forking AgentScope	2026-04-13	v2.0 instability, dual-stack friction, Mode system absent, tool system covered by Vercel AI SDK
Turborepo monorepo	2026-04-13	Clean package boundaries, shared types, single dev command
Vercel AI SDK for LLM	2026-04-13	Multi-provider, streaming, structured output, tool use, actively maintained
Socket.io for realtime	2026-04-13	Mature, room-based broadcasting, reconnection, fallback to polling
Postgres + pgvector for storage	2026-04-13	Supabase hosting, vector search for long-term memory, no separate vector DB
Phase 1 = Roundtable (not Werewolf)	2026-04-13	Simplest mode validates platform core; market-validated by Accio Work demos